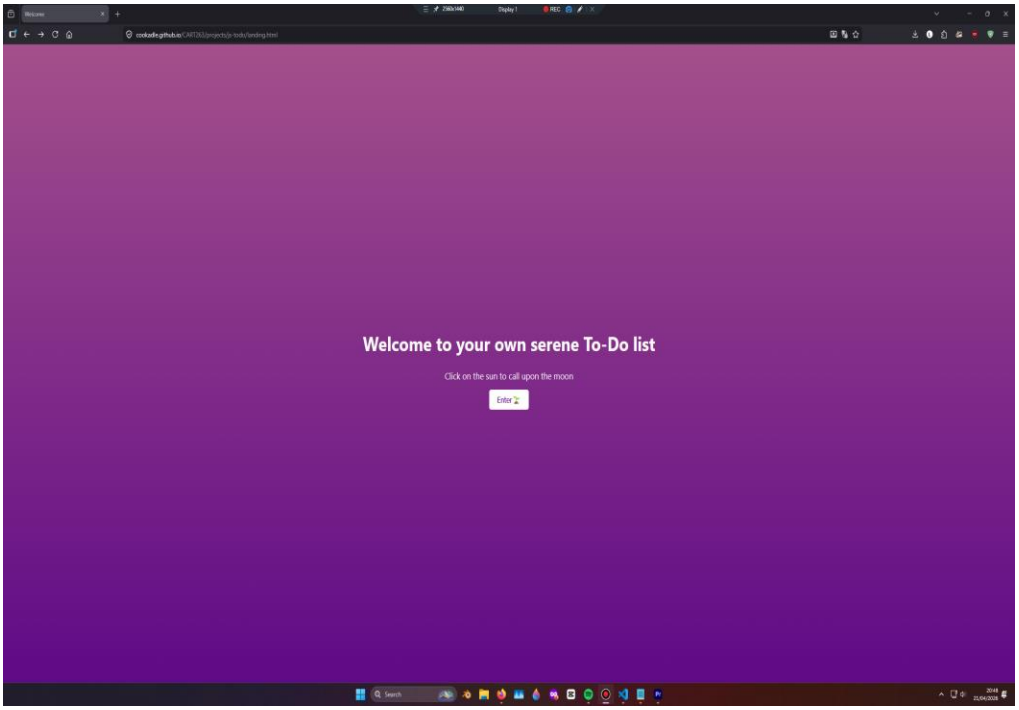
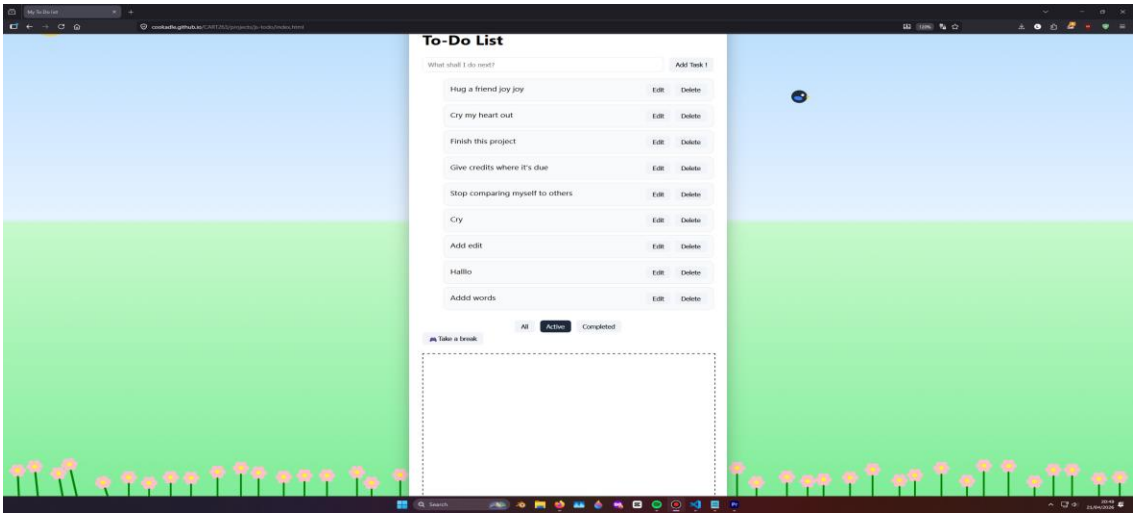
Landing Page-Greeting to users



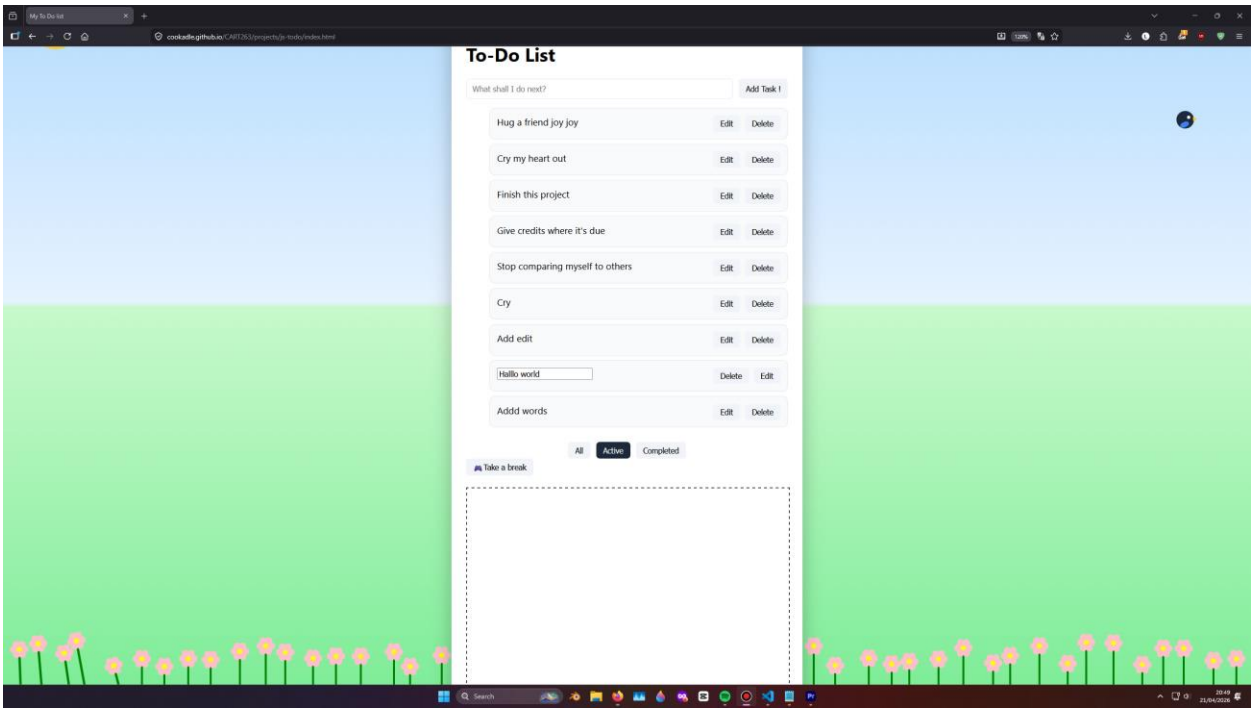
This is my landing page where the user is greeted and given a small instruction about the night mode.

I think that for a web page meant to imitate an app, having a greeting screen before entering the to-do list interface helps set the tone. The separation between the landing page and the to-do page frames the experience as something more intentional rather than just a standard productivity tool, which is something many apps also do and somethingI was told to work on.

To Do List –Tasks and manipulation

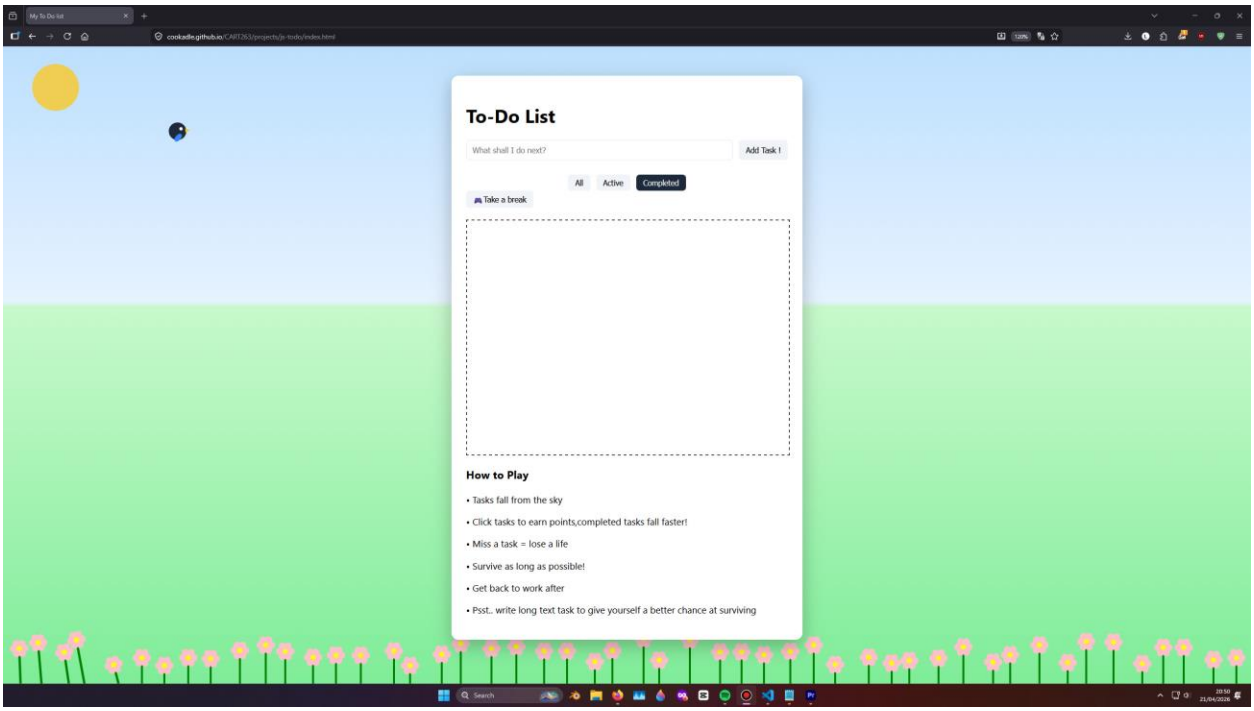


We are now taken to the core functionality of the application: a dynamic to-do list. Tasks are created and displayed using DOM manipulation, which allow users to manage their activities live. Each task exists as an interactive element that supports completion toggling upon clicking on them, editing, and the option to delete them. I also integrated persistent storage using local storage, ensuring that tasks remain available across sessions even if we are to refresh the page.

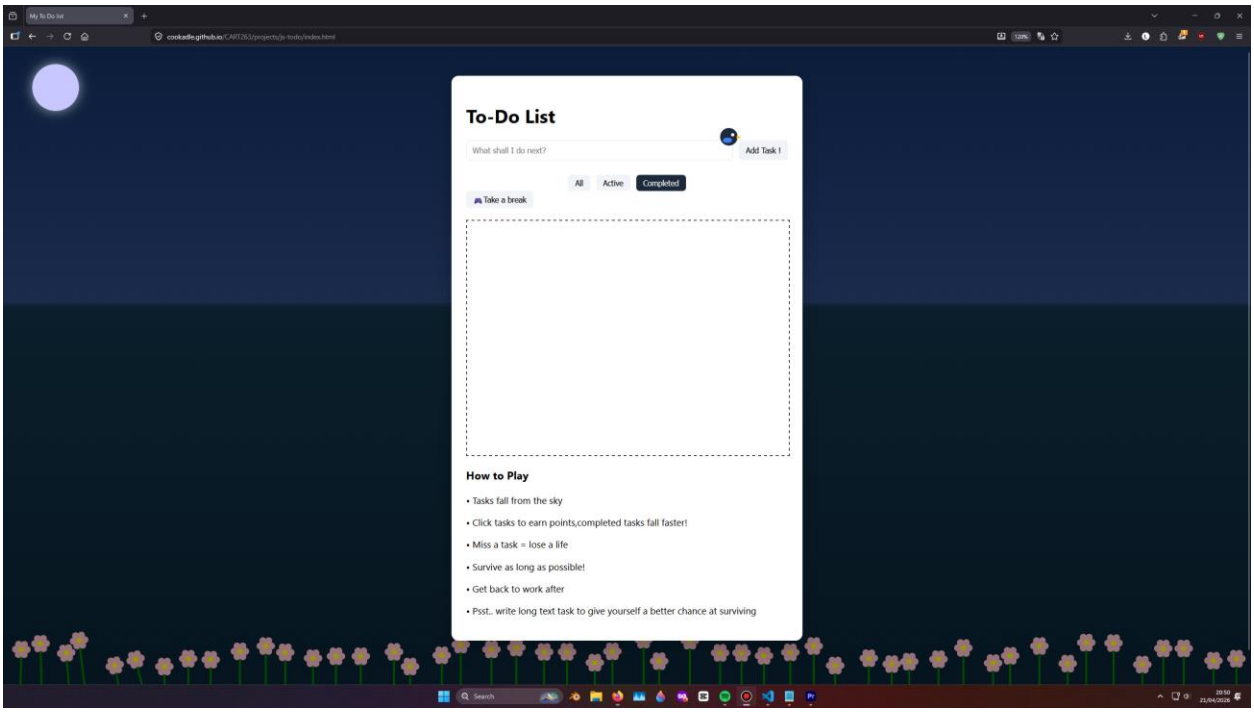


Here is the edit function, where upon clicking the edit button the task gets replaced by an input field, similar to when adding a new task. This allows the user to directly modify the content.

Garden Time – Sun and Moon

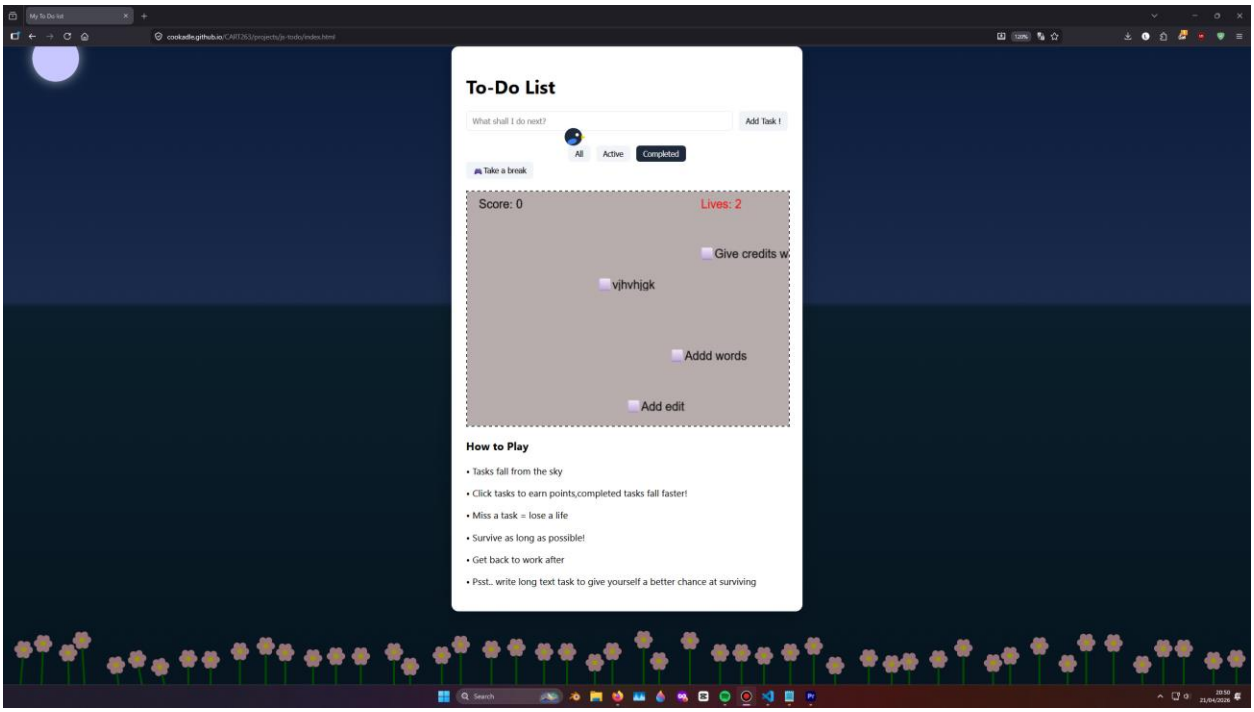


I configured a garden due to my lack of “pizzazz” in the first part of my project. Since the day mode uses bright and jolly colours, I added flowers to create a calm and welcoming atmosphere. The flowers, originally from Task 4, were adapted into my garden. They are randomly generated using a loop, which distributes them across the screen with varying heights and positions. They also react to mouse interaction using inertia-based movement, which makes the environment feel more alive.



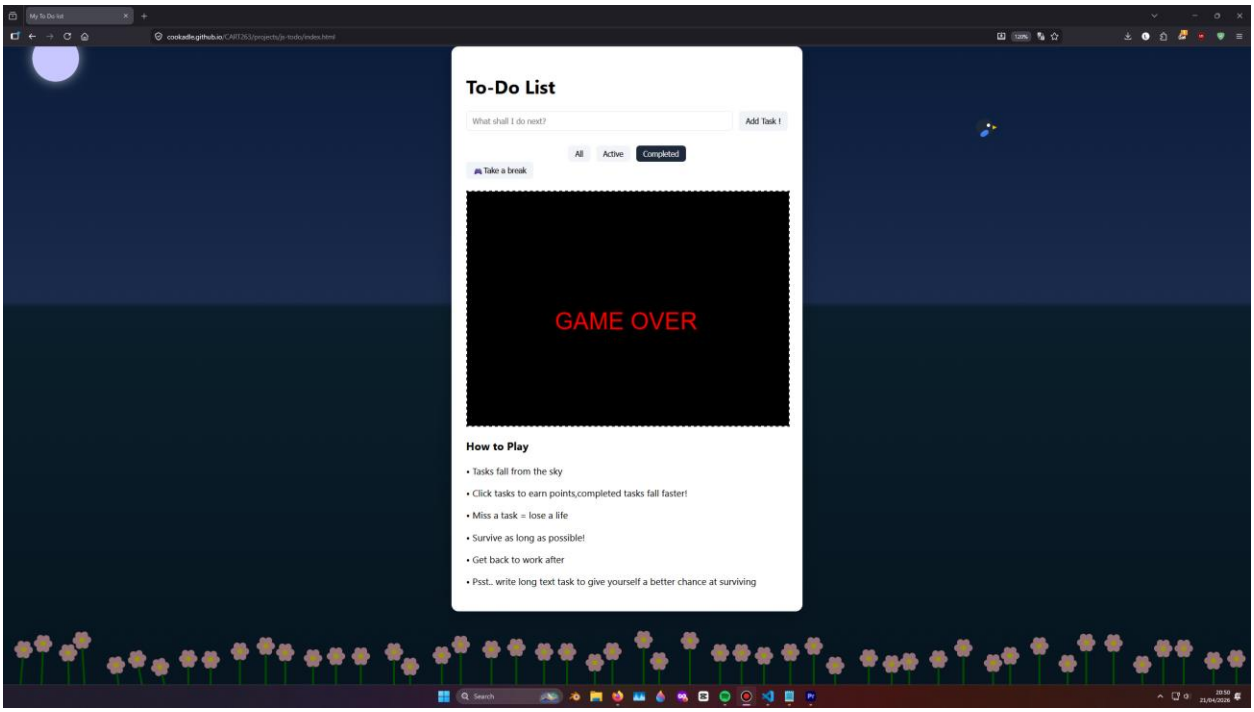
After clicking on the sun, the interface shifts into a darker visual style, which represents my version of night mode. This is achieved through CSS by altering colours and lighting to mimic a different time of day while keeping the same garden elements. I chose not to create a completely new scene, but instead transform the existing one. This keeps the design consistent while still adding variation and interaction, allowing the user to influence their environment.

Phaser Game



This is my integration of the Phaser 3 game engine. Tasks from the to-do list are passed into the game and transformed into falling objects. For me, reusing the task data was a fun way to incorporate user-personalized elements into gameplay.

This means that no game session is ever exactly the same, since the tasks depend on what the user has written. Each falling object retains properties from the original task, such as its completion state, which influences its behavior and speed.



The game over screen marks the end of the user gaming session. If the user misses too many tasks, they lose all lives and the game ends. Ideally, this acts as a short break before returning to productivity.

Reflection/Intent

The project is built using HTML, CSS, and JavaScript. The to-do list system relies heavily on DOM manipulation to dynamically create and update tasks. Toward the end it was heavily CCS modified due to me giving up the idea of using Tailwinds as a library and instead focus on working with the current CSS

Truthfully, I initially had more ambitious plans for this project and wanted to push it closer to a fully developed web-based app. However, after feedback noting that the project lacked “pazzazz,” I shifted my focus.

I had to remind myself I am not a computer science bro and computation art is not just about functionality, but also about interaction, creativity, and playfulness. As a result, the project evolved into something that combines productivity, animation, and gameplay.Now, with this feedback the intention behind this project became the exploration of how a traditional to-do list could become more engaging and less repetitive.

The final result being an interface where tasks are not only functional elements, but also visual and interactive components. They exist both as things to complete and as elements that can be experienced (through the game), redefining the relationship between the user and their tasks.

References / Inspirations(All sources are linked in brainstorming and in the code itself)

https://github.com/Cookadle/CART263/blob/main/projects/js-todo/Brainstormings.md

1. JavaScript To-Do List with Local Storage

I was inspired by basic JavaScript to-do list examples that use local storage to save tasks. These helped me understand how to structure task objects and manage adding, editing, and deleting items dynamically. This became the foundation of my project's core system.

As you can see in my brainstorming page I listed many sources. One thing about my code is that they are a bit Frankenstein-esque which mean I take inspiration from many code with same function but adapt it with concept I know and feel comfortable using while being able to explain the how of the code. All those code snippets are combine into something that works out for me and that I can understand.

2. Phaser 3 Tutorials and Examples (Pippin and the likes)

Phaser 3 tutorials helped guide the structure of my game system, especially in terms of creating scenes, adding physics, and handling interaction. The idea of spawning falling objects and tracking score and lives was influenced by these examples, which I adapted to work with my task system to make my own.

At first, I felt like I wasn't using much of the class resources I was taught, but I realized I used quite a good amount, they just combine neatly that it doesn't feel like much. I got to use DOM manipulation and event handling. I used local storage for the data storage of the task. A handful amount of CSS animations and transitions which then get combined with request animation frame. Then I use Phaser 3 as a library and configured arcade physics for movement and interaction in the falling task game. Despite not using 3D elements, I am proud of my work just a bit too shy to add it on the discord channel sadly.